

Modul Praktikum

Algoritma dan Struktur Data (TPL2106)

Oleh:

Dr. Sofiyanti Indriasari, S.Kom., M.Kom.

**Teknologi Rekayasa Perangkat Lunak
Semester Genap
Tahun Akademik 2025/2026**

Modul 9

Graph III: Spanning Trees (Kruskal's & Prim's Approaches)

1. Deskripsi Singkat Modul

Modul ini membahas konsep **Spanning Tree** pada graph, yaitu subgraph yang menghubungkan seluruh node tanpa membentuk cycle (siklus). Fokus utama modul adalah memahami bagaimana memilih edge-edge terbaik sehingga seluruh node tetap terhubung dengan total bobot minimum.

Pada modul ini, mahasiswa akan mempelajari konsep **Minimum Spanning Tree (MST)** serta dua algoritma yang umum digunakan untuk membentuk MST, yaitu:

- **Kruskal's Algorithm**
- **Prim's Algorithm**

Mahasiswa juga akan memahami perbedaan pendekatan kedua algoritma tersebut, bagaimana proses pemilihan edge dilakukan, serta implementasinya menggunakan Python.

Konsep spanning tree banyak digunakan dalam:

- pembangunan jaringan komputer,
- desain jaringan listrik,
- optimasi jalur distribusi,
- pembangunan infrastruktur,
- dan perancangan koneksi minimum biaya.

2. Capaian Pembelajaran

Setelah mengikuti praktikum pada modul ini, mahasiswa diharapkan mampu:

1. Menjelaskan konsep spanning tree.
2. Memahami konsep minimum spanning tree (MST).
3. Mengidentifikasi cycle pada graph.
4. Memahami prinsip kerja algoritma Kruskal.
5. Memahami prinsip kerja algoritma Prim.
6. Mengimplementasikan Kruskal dan Prim menggunakan Python.
7. Membandingkan pendekatan kedua algoritma.
8. Menentukan MST dengan total bobot minimum.

3. Prasyarat Pengetahuan Awal

Sebelum mengikuti praktikum pada modul ini, mahasiswa diharapkan telah memahami:

- Representasi graph
- Weighted graph
- BFS dan DFS
- Shortest path
- Sorting dasar
- List dan dictionary Python

4. Fungsi dan perulangan Kebutuhan Alat

A. Perangkat Keras

Laptop/PC dengan spesifikasi minimum:

- RAM minimal 4 GB (disarankan 8 GB)
- Ruang kosong minimal 1 GB

Koneksi internet (opsional) untuk instalasi awal dan update extension.

B. Perangkat Lunak

- Python 3.x (disarankan Python 3.10+)
- Visual Studio Code (VS Code)
- Extension VS Code: Python (Microsoft)
- Windows Command Prompt / PowerShell / Git Bash

C. Folder kerja praktikum

5. Intruksi Ketentuan Teknis Praktikum

- Setiap latihan disimpan dalam file Python terpisah.
- Program wajib diberi komentar penjelasan.
- Program harus dapat dijalankan tanpa error.
- Mahasiswa wajib memahami proses pembentukan spanning tree, bukan hanya menjalankan program.
- Penamaan file mengikuti instruksi modul

Praktikum13.materi1.py s/d praktikum13.Materi2.py

Praktikum13.latihan1.py s/d praktikum13.latihan5.py

- Setiap kode program harus diberi **komentar** untuk menjelaskan tujuan blok program sebagai pembiasaan dokumentasi kode. Serta gunakan komentar pada program terkait untuk menjawab pertanyaan-pertanyaan pada latihan

6. Materi

6.1 Konsep Spanning Tree

Pada graph yang saling terhubung, sering kali terdapat banyak kemungkinan jalur yang dapat digunakan untuk menghubungkan seluruh node. Namun, dalam beberapa kasus, tidak semua edge diperlukan. Jika terlalu banyak edge digunakan, graph dapat membentuk cycle (siklus) dan menyebabkan penggunaan biaya yang lebih besar.

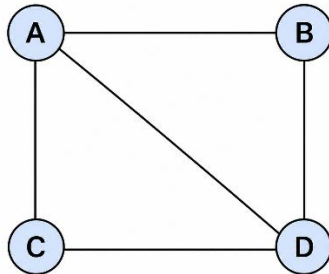
Untuk mengatasi hal tersebut, digunakan konsep **Spanning Tree**.

Spanning Tree adalah subgraph dari sebuah graph yang:

- menghubungkan seluruh node,
- tidak memiliki cycle,
- dan memiliki jumlah edge sebanyak:
 $\text{jumlah node} - 1$

Artinya, jika terdapat 5 node, maka spanning tree hanya memiliki 4 edge..

Ilustrasi graph awal

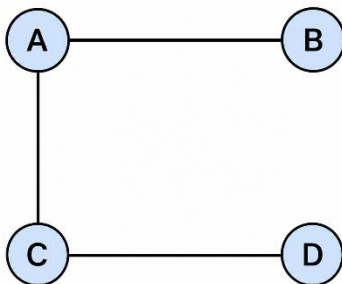


Graph di atas memiliki beberapa kemungkinan jalur dan mengandung cycle. **Mengapa Cycle Dihindari?** Dalam spanning tree, cycle dihindari karena:

- menyebabkan penggunaan edge berlebih,
- meningkatkan biaya total,
- membuat koneksi tidak efisien.

Bayangkan membangun jaringan kabel internet antar gedung. Jika semua gedung sudah saling terhubung, maka menambahkan kabel tambahan yang membentuk lingkaran akan meningkatkan biaya tanpa benar-benar diperlukan.

Contoh Spanning Tree



Graph diatas adalah spanning tree karena :

- seluruh node tetap terhubung
- tidak ada cycle
- jumlah edge lebih sedikit

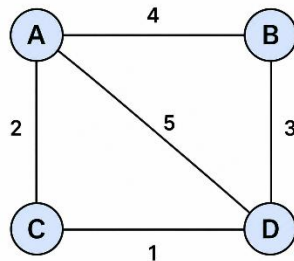
Jadi Spanning Tree bukan berarti mencari jalur terpendek antar node, tetapi: mencari cara menghubungkan seluruh node tanpa siklus.

6.2 Minimum Spanning Tree (MST)

Minimum Spanning Tree (MST) adalah sebuah spanning tree pada weighted graph yang menghubungkan seluruh node tanpa membentuk cycle dan memiliki total bobot minimum dibandingkan kemungkinan spanning tree lainnya. MST digunakan untuk memperoleh koneksi paling efisien dengan biaya terkecil, sehingga sangat berguna pada kasus seperti

pembangunan jaringan komputer, instalasi kabel listrik, jalur distribusi, dan infrastruktur transportasi.

Perhatikan graph berikut ini :



Kemungkinan MST. Jika dipilih edge:

A - C = 2

C - D = 1

D - B = 3

Maka total bobot : $2 + 1 + 3 = 6$

Seluruh node tetap terhubung dan tidak membentuk cycle.

Jadi Minimum Spanning Tree bertujuan:

- menghubungkan seluruh node
- tanpa cycle
- dengan total bobot minimum

6.3 Algoritma Kruskal

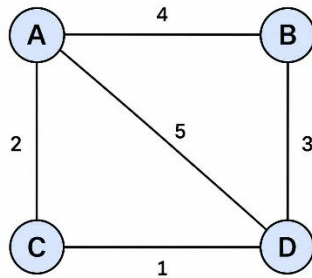
Algoritma Kruskal adalah algoritma Minimum Spanning Tree (MST) yang bekerja dengan memilih edge dengan bobot paling kecil terlebih dahulu. Proses pemilihan edge dilakukan secara bertahap berdasarkan urutan bobot terkecil hingga seluruh node terhubung. Namun, sebuah edge hanya akan ditambahkan ke dalam MST jika tidak membentuk cycle, sehingga graph tetap terhubung secara efisien tanpa menghasilkan siklus.

Langkah Algoritma Kruskal

1. Urutkan seluruh edge berdasarkan bobot terkecil.
2. Pilih edge dengan bobot terkecil.
3. Periksa apakah edge membentuk cycle.
4. Jika tidak membentuk cycle, tambahkan ke MST.
5. Jika membentuk cycle, abaikan.
6. Ulangi sampai seluruh node terhubung.



Contoh , perhatikan graph berikut :



Urutkan berdasarkan bobot :

Edge	Bobot
C-D	1
A-C	2
B-D	3
A-B	4
A-D	5

Proses Pemilihan

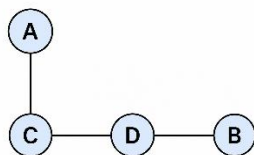
Pilih C-D : aman

Pilih A-C : aman

Pilih B-D : aman

Semua node sudah terhubung.

Hasil MST :



Total Bobot : $2 + 1 + 3 = 6$

Implementasi pada Phyton:

```
# =====  
# Implementasi Kruskal  
# =====  
  
# Daftar edge: (bobot, node1, node2)  
edges = [  
    (1, 'C', 'D'),  
    (2, 'A', 'C'),  
    (3, 'B', 'D'),  
    (4, 'A', 'B'),  
    (5, 'A', 'D')  
]
```

```
# Mengurutkan edge berdasarkan bobot
edges.sort()

mst = []
total_weight = 0

# Set sederhana untuk node yang sudah dipilih
connected = set()

for weight, u, v in edges:

    # Jika edge tidak membentuk cycle sederhana
    if u not in connected or v not in connected:

        mst.append((u, v, weight))
        total_weight += weight

        connected.add(u)
        connected.add(v)

print("Minimum Spanning Tree:")

for edge in mst:
    print(edge)

print("Total bobot =", total_weight)
```

Kelemahan algoritma Kruskal adalah proses pendeteksian cycle dapat menjadi lebih kompleks pada graph yang besar, sehingga biasanya memerlukan struktur data tambahan seperti Union-Find atau Disjoint Set agar lebih efisien. Selain itu, karena Kruskal bekerja dengan mengurutkan seluruh edge terlebih dahulu, proses sorting dapat memerlukan waktu komputasi yang cukup besar jika jumlah edge sangat banyak. Pada graph yang sangat padat (dense graph), algoritma Kruskal juga cenderung kurang efisien dibandingkan algoritma Prim karena harus memproses lebih banyak edge secara global.

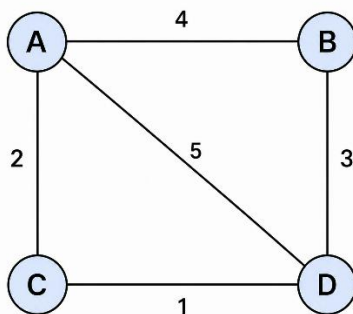
6.4 Algoritma Prim

Algoritma Prim adalah algoritma Minimum Spanning Tree (MST) yang bekerja dengan membangun spanning tree secara bertahap mulai dari satu node awal. Pada setiap langkah, algoritma akan memilih edge dengan bobot paling kecil yang menghubungkan node yang sudah berada di dalam tree dengan node lain yang belum terhubung. Proses ini terus dilakukan hingga seluruh node pada graph saling terhubung tanpa membentuk cycle. Berbeda dengan Kruskal yang berfokus pada pemilihan edge secara global, Prim lebih berorientasi pada pengembangan tree dari node awal ke node-node di sekitarnya.

Langkah Algoritma Prim

1. Pilih node awal.
2. Cari edge terkecil dari node yang sudah dikunjungi.
3. Tambahkan node baru ke MST.
4. Ulangi sampai semua node terhubung.

Contoh : perhatikan graph berikut ini



Mulai dari A. **Edge terkecil dari A:**

Edge	Bobot
A-C	2
A-B	4
A-D	5

Pilih A-C

Node berikutnya :

Sekarang node aktif: A , C

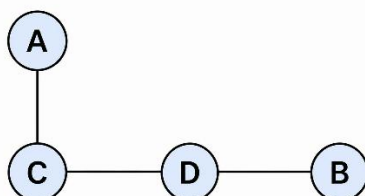
Edge minimum berikutnya : C-D = 1

Lanjut

Node aktif : A, C, D

Edge minimum berikutnya : D-B = 3

Hasil MST





Implementasi pada Python:

```
import heapq

graph = {
    'A': {'B': 4, 'C': 2, 'D': 5},
    'B': {'A': 4, 'D': 3},
    'C': {'A': 2, 'D': 1},
    'D': {'A': 5, 'B': 3, 'C': 1}
}

def prim(graph, start):

    visited = set([start])

    edges = []

    for neighbor, weight in graph[start].items():
        heapq.heappush(edges, (weight, start, neighbor))

    mst = []
    total_weight = 0

    while edges:

        weight, u, v = heapq.heappop(edges)

        if v not in visited:

            visited.add(v)

            mst.append((u, v, weight))
            total_weight += weight

            for neighbor, w in graph[v].items():

                if neighbor not in visited:
                    heapq.heappush(edges, (w, v, neighbor))

    return mst, total_weight

mst, total = prim(graph, 'A')

print("Minimum Spanning Tree:")

for edge in mst:
    print(edge)

print("Total bobot =", total)
```

Algoritma Kruskal dan Prim merupakan dua pendekatan populer untuk membentuk Minimum Spanning Tree (MST) pada weighted graph. Keduanya memiliki tujuan yang sama, yaitu menghubungkan seluruh node dengan total bobot minimum tanpa membentuk cycle, namun menggunakan strategi yang berbeda. Kruskal bekerja dengan memilih edge berbobot terkecil secara global, sedangkan Prim membangun tree secara bertahap mulai dari satu node awal. Dengan memahami kedua algoritma ini, mahasiswa dapat mempelajari bagaimana graph dapat dioptimalkan untuk menghasilkan koneksi yang efisien dan memahami bahwa pemilihan algoritma dapat disesuaikan dengan karakteristik graph yang digunakan.

Perbandingan Kruskal dan Prim

Aspek	Kruskal	Prim
Pendekatan	Memilih edge global terkecil	Membesar dari node awal
Fokus	Edge	Node
Cocok untuk	Sparse graph	Dense graph

7. Latihan - Latihan

Bagian ini merupakan inti kegiatan praktikum. Mahasiswa diminta membaca ulang materi pada Bagian 6, menjalankan kode program yang tersedia, mengamati output, menjawab pertanyaan analisis dalam bentuk komentar pada file program, serta mengembangkan program sesuai instruksi soal. Setiap latihan disimpan dalam file Python terpisah dengan format penamaan:

- praktikum13.latihan1.py
- praktikum13.latihan2.py
- praktikum13.latihan3.py
- praktikum13.latihan4.py
- praktikum13.latihan5.py

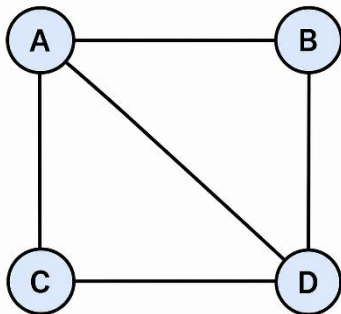
Setiap file wajib diberi header identitas mahasiswa di bagian atas program:

```
# Nama :  
# NIM :  
# Kelas :  
# Praktikum 13 - Graph III: Spanning Tree
```

Mahasiswa wajib menuliskan jawaban analisis sebagai komentar di bagian bawah file program.

Latihan 1 . Memahami Konsep Spanning Tree

Perhatikan graph berikut ini :



Graph tersebut memiliki beberapa kemungkinan spanning tree.

Buatlah program Python sederhana yang:

1. Menampilkan daftar edge pada graph.
2. Menampilkan contoh spanning tree yang valid.
3. Menampilkan jumlah edge pada graph awal.
4. Menampilkan jumlah edge pada spanning tree.

Gunakan list atau dictionary sederhana untuk menyimpan data edge.

Contoh Struktur Program

```
# Daftar edge graph
edges = [
    ('A', 'B'),
    ('A', 'C'),
    ('A', 'D'),
    ('C', 'D'),
    ('B', 'D')
]

# Contoh spanning tree
spanning_tree = [
    ('A', 'C'),
    ('C', 'D'),
    ('D', 'B')
]

print("Edge pada graph:")
for edge in edges:
    print(edge)

print("\nSpanning Tree:")
```

```
for edge in spanning_tree:
    print(edge)

print("\nJumlah edge graph =", len(edges))
print("Jumlah edge spanning tree =", len(spanning_tree))
```

Pertanyaan Analisis

Tuliskan jawaban sebagai komentar di bagian bawah program.

```
# Jawaban Analisis:
# 1. Apa perbedaan graph awal dan spanning tree?
# 2. Mengapa spanning tree tidak boleh memiliki cycle?
# 3. Mengapa jumlah edge spanning tree selalu lebih sedikit?
```

Latihan 2 . Implementasi Algoritma Kruskal

Salin dan jalankan program berikut. Amati hasil output dan urutan edge yang dipilih oleh algoritma Kruskal.

```
# =====
# Implementasi Sederhana Algoritma Kruskal
# =====

# Daftar edge: (bobot, node1, node2)
edges = [
    (1, 'C', 'D'),
    (2, 'A', 'C'),
    (3, 'B', 'D'),
    (4, 'A', 'B'),
    (5, 'A', 'D')
]

# Mengurutkan edge berdasarkan bobot terkecil
edges.sort()

mst = []
total_weight = 0

connected = set()

for weight, u, v in edges:

    # Memilih edge yang tidak membentuk cycle sederhana
```

if u not in connected or v not in connected:

```
mst.append((u, v, weight))
total_weight += weight

connected.add(u)
connected.add(v)

print("Minimum Spanning Tree:")

for edge in mst:
    print(edge)

print("Total bobot =", total_weight)
```

Pertanyaan Analisis

- # Jawaban Analisis:
- # 1. Edge mana yang dipilih pertama kali?
- # 2. Mengapa edge dengan bobot paling kecil dipilih lebih dahulu?
- # 3. Berapa total bobot MST yang dihasilkan?
- # 4. Mengapa edge tertentu tidak dipilih?

Latihan 3 . Implementasi Algoritma Prim

Salin dan jalankan program berikut. Amati bagaimana algoritma Prim membangun MST secara bertahap mulai dari node awal.

```
import heapq

graph = {
    'A': {'B': 4, 'C': 2, 'D': 5},
    'B': {'A': 4, 'D': 3},
    'C': {'A': 2, 'D': 1},
    'D': {'A': 5, 'B': 3, 'C': 1}
}

def prim(graph, start):
    visited = set([start])
    edges = []

    for neighbor, weight in graph[start].items():
        heapq.heappush(edges, (weight, start, neighbor))

    mst = []
    total_weight = 0

    while edges:
```

```
weight, u, v = heapq.heappop(edges)

if v not in visited:

    visited.add(v)

    mst.append((u, v, weight))
    total_weight += weight

    for neighbor, w in graph[v].items():

        if neighbor not in visited:
            heapq.heappush(edges, (w, v, neighbor))

return mst, total_weight

mst, total = prim(graph, 'A')

print("Minimum Spanning Tree:")

for edge in mst:
    print(edge)

print("Total bobot =", total)
```

Pertanyaan Analisis:

- # Jawaban Analisis:
- # 1. Node awal apa yang digunakan?
- # 2. Edge mana yang dipilih pertama kali?
- # 3. Bagaimana Prim menentukan edge berikutnya?
- # 4. Berapa total bobot MST yang dihasilkan?
- # 5. Apa perbedaan pendekatan Prim dan Kruskal?

Latihan 4. Studi Kasus: Jaringan Kabel Antar Gedung

Deskripsi Kasus

Sebuah kampus ingin membangun jaringan kabel internet antar gedung dengan biaya minimum. Setiap hubungan antar gedung memiliki biaya pemasangan kabel yang berbeda. Data hubungan gedung:

GedungA - GedungB = 4
GedungA - GedungC = 2
GedungB - GedungD = 3
GedungC - GedungD = 1
GedungA - GedungD = 5



Buatlah program menggunakan algoritma Prim atau Kruskal untuk menentukan jaringan kabel dengan total biaya minimum. Ketentuan Program harus memuat:

- Representasi weighted graph.
- Implementasi Prim atau Kruskal.
- Output edge yang dipilih.
- Output total biaya minimum.
- Komentar penjelasan program.

Pertanyaan Analisis:

- # Jawaban Analisis:
- # 1. Algoritma apa yang digunakan?
- # 2. Edge mana saja yang dipilih?
- # 3. Berapa total biaya minimum?
- # 4. Mengapa MST cocok digunakan pada kasus ini?

Latihan 5 . Tugas Mandiri: Buat Program MST dengan Kasus Baru

Buatlah program Minimum Spanning Tree menggunakan algoritma Kruskal atau Prim untuk salah satu kasus berikut:

Pilihan Kasus

Kasus 1 . Jaringan Jalan Antar Kota

Bogor - Jakarta = 5
Bogor - Depok = 2
Depok - Jakarta = 3
Jakarta - Bandung = 6
Depok - Bandung = 4

Kasus 2 . Jaringan Komputer

RouterA - RouterB = 3
RouterA - RouterC = 2
RouterB - RouterD = 5
RouterC - RouterD = 1
RouterB - RouterC = 4

Ketentuan Program. Program harus memuat:

1. Representasi weighted graph.
2. Implementasi Kruskal atau Prim.

3. Output MST.
4. Output total bobot minimum.
5. Komentar penjelasan program.

Pertanyaan Analisis:

```
# Jawaban Analisis:  
# 1. Kasus apa yang dipilih?  
# 2. Algoritma apa yang digunakan?  
# 3. Edge mana saja yang dipilih dalam MST?  
# 4. Berapa total bobot MST?  
# 5. Mengapa edge tertentu tidak dipilih?
```

Ketentuan Praktikum

Buatlah semua program latihan pada bagian 6 dan bagian 7 dan masing-masing disimpan dalam file dengan nama

Program Bagian 6 Materi :

Praktikum13.materi1.py s/d praktikum13.Materi2.py

Program Bagian 7 Latihan

Praktikum13.latihan1.py s/d praktikum13.latihan5.py

Setiap file diberikan header identitas mahasiswa pada bagian atas kode program. Diskusikan kemudian tulis penjelasan dan hasil diskusi pada file program sebagai komentar. Semakin jelas dan lengkap akan semakin bagus. Kumpulkan di akhir praktikum di repo masing-masing pada **folder praktikum13**.